VNU-HCM

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Seminar Report: Generative Artificial Intelligence

Topic: Diffusion Models

Course: CS420: Introduction to Artificial Intelligence

Students:
Vong Chi Van - 24125049
Le Dat Phuc An - 24125052

November 10, 2026



[S06] Diffusion Models: From Adding Noise to Generating Images

Theoretical Foundations, Step-by-Step Worked Derivations, and Latent Scaling

Vong Chi Van Le Dat Phuc An
Faculty of Information Technology, University of Science, VNU-HCM
{24125049, 24125052}@student.hcmus.edu.vn

Abstract

Diffusion models have emerged as a dominant paradigm in generative artificial intelligence, achieving unprecedented visual fidelity and robust likelihood-based training without adversarial instability. This report investigates the fundamental guiding question: *how can learning to remove noise enable a neural network to generate novel images, and why does diffusion within a learned latent space drastically reduce computational requirements?* Grounded in two foundational papers, Denoising Diffusion Probabilistic Models (Ho et al., 2020) and Latent Diffusion Models (Rombach et al., 2022), we examine the mathematical principles of forward Gaussian corruption, reverse denoising, and perceptual latent compression. Through theoretical derivations, a step-by-step numerical worked example, and empirical benchmark experiments, we provide an integrated analysis of diffusion dynamics, high-resolution scaling trade-offs, and practical deployment limitations.

Keywords: Generative AI, Denoising Diffusion Probabilistic Models, Latent Diffusion Models, Variational Autoencoders, Cross-Attention, Generative Learning Trilemma.

1 Introduction

Synthesizing high-dimensional continuous signals such as natural images is a central challenge in artificial intelligence. A natural image tensor is represented as $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, where H, W, and C denote the height, width, and color channel dimensions, respectively. Unlike supervised discriminative learning, which only seeks decision boundaries separating predefined class labels, a generative model must capture the true underlying probability distribution $p_{\text{data}}(\mathbf{x})$ of a complex, low-dimensional data manifold embedded in high-dimensional ambient space. Once learned, the model enables the synthesis of novel, realistic samples $\mathbf{x}_{\text{new}} \sim p_\theta(\mathbf{x})$.

Deep generative architectures have historically struggled to simultaneously achieve high sample sharpness, comprehensive mode diversity, and fast sampling speed. While early generative paradigms made substantial progress, each faced inherent trade-offs between sample quality, training stability, and sampling efficiency.

Guiding Question and Scope. *How can learning to remove noise enable a model to generate new images, and why does moving diffusion into a learned latent space resolve its computational bottleneck?*

The core insight of Denoising Diffusion Probabilistic Models (DDPM) [3, 1] is to decompose the difficult one-step mapping from standard Gaussian noise $\mathcal{N}(\mathbf{0}, \mathbf{I})$ to the data distribution $p_{\text{data}}(\mathbf{x}_0)$ into T small, tractable steps ($T \approx 1000$). If a fixed forward process gradually corrupts a clean image $\mathbf{x}_0$ with small increments of Gaussian noise until the signal is erased into pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then learning the reverse process allows a neural network to start from random Gaussian noise and iteratively sculpt a realistic image through repeated denoising steps:

$$\mathbf{x}_T \to \mathbf{x}_{T-1} \to \cdots \to \mathbf{x}_0 \tag{1}$$

Furthermore, to overcome the immense computational cost of evaluating a full-resolution neural network in pixel space, Latent Diffusion Models (LDM) [2] first compress images into the compact latent space of a pretrained Variational Autoencoder before executing diffusion. This representation reduces the state dimensionality by $48\times$ and cuts self-attention complexity by $4{,}096\times$, rendering high-resolution synthesis tractable on standard consumer hardware.

Tasks and Report Organization. To systematically address the guiding question in accordance with the seminar requirements, this report is structured around six concrete tasks:

- *Task 1: Foundational Paradigms and the Generative Trilemma.* We contrast discriminative classifiers with generative modeling and analyze autoregressive models, variational autoencoders, generative adversarial networks, and diffusion models across the generative learning trilemma (Sections 2.1 and 2.2).
- *Task 2: DDPM Mathematical Framework.* We formulate the variance-preserving forward process, variance schedule selection, closed-form multi-step transitions, the learned reverse Markov chain, and the simplified noise-prediction training objective (Section 3.1).
- *Task 3: Symbol Definitions and 1D Worked Example.* We specify complete symbol definitions, physical units, and modeling assumptions, accompanied by a step-by-step one-dimensional numerical worked example illustrating stochastic signal contraction and reverse denoising (Section 3.2).
- *Task 4: Multi-Level Forward Corruption and Reverse Image Generation.* We demonstrate forward noise corruption across multiple timesteps and iterative reverse image generation trajectories on empirical benchmarks (Section 4.2).
- *Task 5: Latent Diffusion Architecture.* We analyze the two-stage inductive bias of Latent Diffusion Models, examining perceptual autoencoder compression and cross-attention conditioning mechanisms for multimodal guidance (Section 3.3).
- *Task 6: Pixel vs. Latent Comparative Evaluation and Practical Limitations.* We provide an evidence-supported comparison between pixel-space and latent-space diffusion across explicit computational and quality metrics, followed by a critical evaluation of real-world limitations including sequential sampling latency, prompt attribute binding, copyright memorization, and societal bias (Sections 4.3 and 5).

2 Background and Paper Review

2.1 Distinguishing Discriminative vs. Generative Models

Supervised machine learning and probabilistic modeling are fundamentally divided into two complementary paradigms:

1. *Discriminative Models:* Discriminator models are mainly used for classification and regression tasks. A discriminative model directly learns to predict the conditional probability distribution:

$$P(Y \mid \mathbf{x}) \tag{2}$$

   where Y is the class label and $\mathbf{x}$ represents the input features. It learns decision boundaries separating classes without modeling the distribution of the input features $\mathbf{x}$. Consequently, a discriminative model cannot generate novel data samples.

2. *Generative Models:* Generative models are primarily used to generate novel data, though they can also be used for classification. A generative model learns the underlying data distribution:

$$p(\mathbf{x}) \tag{3}$$

   or the joint distribution:

$$p(\mathbf{x}, Y) \tag{4}$$

   when conditioned on class labels Y. Its primary function is synthesizing new, realistic samples:

$$\mathbf{x}_{\text{new}} \sim p(\mathbf{x}) \tag{5}$$

   that resemble the training data distribution.

Figure 1 shows the visual comparison between the two paradigms on a synthetic four-class two-dimensional dataset.

2.2 Deep Generative Paradigms and The Generative Learning Trilemma (Task 1)

To contextualize diffusion models within the generative literature, we examine prior generative families across the Generative Learning Trilemma [17], which posits that a generative model ideally satisfies three core properties:

- *High Sample Quality:* Generating sharp, detailed, and photorealistic samples.

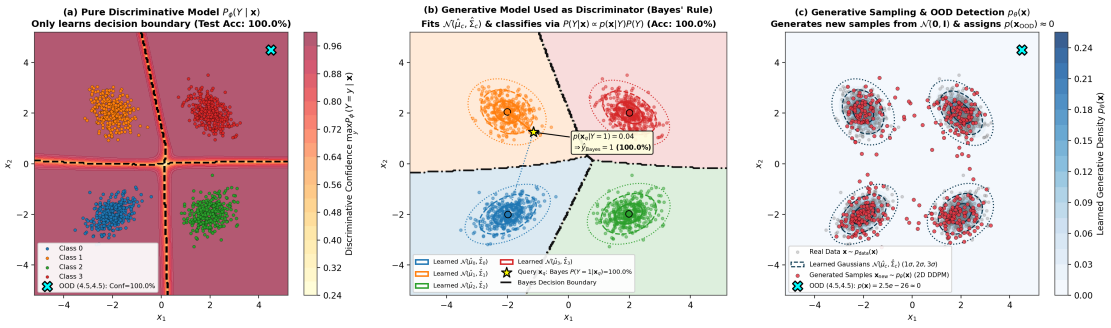


Figure 1. Visual comparison of modeling paradigms on a four-class two-dimensional manifold (Task 1): (a) A discriminative classifier learns quadrant decision boundaries to predict $P(Y \mid \mathbf{x})$ for classification; (b) A generative model learns class-conditional distributions $p(\mathbf{x} \mid Y)$ capturing the shape of the data; (c) The learned generative density enables generating new samples $\mathbf{x}_{\text{new}} \sim p(\mathbf{x})$ and detecting out-of-distribution queries.

- *Fast Sampling:* Synthesizing samples with minimal forward passes (ideally real-time or single-step).
- *Full Mode Coverage (Diversity):* Faithfully covering all modes of the data distribution without mode collapse.

Historically, generative architectures achieved at most two of these three objectives:

1. *Autoregressive Models (PixelRNN, PixelCNN) [11, 12]:* Model images pixel-by-pixel using the probability chain rule:
$$p_\theta(\mathbf{x}) = \prod_{i=1}^{n} p_\theta(x^{(i)} \mid x^{(<i)}) \tag{6}$$
They maximize exact log-likelihood and avoid mode collapse, achieving high quality and full diversity. However, generating an image requires sequential pixel-by-pixel generation, yielding quadratic computational complexity and slow sampling.

2. *Variational Autoencoders (VAEs) [4, 5]:* Introduce an unobserved latent variable $\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and optimize the evidence lower bound (ELBO). VAEs generate images in a single forward decoder pass and cover all modes. However, because pixel likelihood objectives enforce mean-squared or cross-entropy reconstruction, VAEs average across plausible modes and produce visibly blurry samples.

3. *Generative Adversarial Networks (GANs) [6, 7]:* Frame synthesis as a two-player minimax game between a Generator and a Discriminator. GANs produce razor-sharp images in a single feed-forward step. However, because they lack an explicit density objective, GANs suffer from training instability and severe mode collapse.

4. *Denoising Diffusion Probabilistic Models (DDPM) [3, 1]:* Formulate image generation as inverting a parameterized Markov chain. By optimizing a stable variational objective that reduces to mean squared error, diffusion models achieve state-of-the-art visual fidelity and comprehensive mode coverage. Their primary bottleneck is sampling speed, requiring hundreds to thousands of iterative network evaluations.

5. *Latent Diffusion Models (LDM) [2]:* Resolve the sampling and training efficiency limitations of DDPM by operating within a perceptually compressed latent space learned by a pretrained autoencoder, achieving high sample fidelity and faster throughput.

Table 1 summarizes the theoretical formulation, latent structure, and empirical trade-offs of each generative family, citing primary literature sources for every theoretical property and empirical claim.

2.3 Foundational Literature Review

This report analyzes two foundational milestone papers:

- *Required Paper: Ho, Jain and Abbeel (NeurIPS 2020), "Denoising Diffusion Probabilistic Models" (DDPM) [1]:* Demonstrated that diffusion models generate image quality rivaling GANs while retaining likelihood-based mode coverage. The paper made three key contributions: (1) establishing the equivalence between denoising score matching and Markov chain reverse step estimation; (2) reparameterizing the reverse transition mean to predict the injected Gaussian noise vector $\boldsymbol{\epsilon}$; (3) introducing the unweighted simplified training objective $L_{\text{simple}}(\theta)$, which suppresses small-step noise artifacts and prioritizes perceptual visual features.

Table 1. Qualitative comparison of major generative model families across the Generative Learning Trilemma [17], citing primary literature sources for each theoretical property and empirical claim (Task 1).

Model Family	Density $p_\theta(\mathbf{x})$	Training Mechanism & Latent Structure	Sample Quality	Mode Coverage	Sampling Speed
Autoregressive [11, 12] (PixelRNN, PixelCNN)	Explicit, Exact (Exact MLE) [11]	Chain rule $\prod_{i=1}^{n} p_\theta(x^{(i)} \mid x^{(<i)})$ via causal masked convolutions; no global latent $\mathbf{z}$ [11, 12].	Sharp local contrast [11, 17]	Full (No mode collapse; stable MLE) [11, 1]	Extremely Slow ($\mathcal{O}(n)$ sequential pixel evaluations) [11, 14, 17]
VAE [4, 5] (Variational Autoencoder)	Explicit, Approx. (ELBO Bound) [4, 5]	Encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ + Decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$ with $\dim(\mathbf{z}) \ll \dim(\mathbf{x})$; Reparameterization Trick [4].	Blurry (Single-step L_2/BCE mode averaging) [4, 10, 17]	Full, smooth latent manifold [4, 17]	Very Fast (1 Decoder forward pass) [4, 17]
GAN [6, 7] (DCGAN, StyleGAN)	Implicit (Likelihood-free) [6]	Adversarial minimax game $\min_G \max_D$ between Generator $G_\theta(\mathbf{z})$ and Discriminator $D_\phi(\mathbf{x})$ [6, 7].	Razor Sharp (Mode-seeking via adversarial loss) [6, 8, 17]	Poor (Prone to Mode Collapse and minimax instability) [6, 9, 17]	Very Fast (1 Generator forward pass) [6, 17]
Pixel Diffusion [3, 1] (DDPM)	Explicit, Approx. (T-step ELBO) [3, 1]	Fixed T-step Markov chain with $\dim(\mathbf{x}_t) = \dim(\mathbf{x}_0)$; U-Net predicts noise $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ via unweighted MSE L_{simple} [1].	SOTA Sharpness (FID 3.17 on CIFAR-10) [1, 15]	Full (Stable regression; beats GAN mode coverage) [1, 15, 17]	Slow ($T \approx 1000$ sequential full-res U-Net passes) [1, 14, 17]
Latent Diffusion [2] (LDM / Stable Diffusion)	Two-Stage ELBO (VAE + Diffusion) [2]	Compresses $\mathbf{x}_0 \to \mathbf{z}_0$ via frozen Conv-VAE, then runs Cross-Attention U-Net on latent tensor $\mathbf{z}_t$ [2].	SOTA Photorealism at high resolution (FID 5.11 on CelebA-HQ) [2]	Full and flexible multimodal text conditioning [2]	Fast and Practical (Operates on $48\times$ smaller $\mathbf{z}_t$; 20 to 50 DDIM steps) [2, 14]

- *Recommended Paper: Rombach et al. (CVPR 2022), "High-Resolution Image Synthesis With Latent Diffusion Models" (LDM) [2]:* Identified that training pixel-space diffusion models on high-resolution images allocates excessive compute to imperceptible high-frequency details. Rombach et al. introduced a two-stage paradigm: (1) a first stage Variational Autoencoder (VAE) trained with perceptual and adversarial losses that compresses high-dimensional images into an efficient latent space; (2) a second stage diffusion backbone operating exclusively on latent representations, incorporating cross-attention layers to enable flexible conditioning on text prompts, bounding boxes, or segmentation maps.

3 Method and Step-by-Step Worked Example

3.1 Forward Process, Variance Schedule, Reverse Denoising, and Sampling (Task 2)

Forward Noising Process. Let clean data follow an unknown distribution:

$$\mathbf{x}_0 \sim q(\mathbf{x}_0) \tag{7}$$

The forward diffusion process is a non-trainable Markov chain governed by a fixed variance schedule:

$$\beta_1, \beta_2, \ldots, \beta_T \in (0, 1) \tag{8}$$

The joint distribution over all intermediate steps is defined as:

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \tag{9}$$

where each individual forward transition is Gaussian:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t \mathbf{I}\big) \tag{10}$$

Variance-Preserving Property. Equation (10) implements a Variance-Preserving (VP) stochastic differential process. By scaling the incoming signal by $\sqrt{1-\beta_t}$, the total variance is preserved. Specifically, if the previous state has unit variance:

$$\text{Var}(\mathbf{x}_{t-1}) = \mathbf{I} \tag{11}$$

then the variance of the next state remains unit:

$$\text{Var}(\mathbf{x}_t) = (\sqrt{1-\beta_t})^2\text{Var}(\mathbf{x}_{t-1}) + \beta_t\mathbf{I} = (1-\beta_t)\mathbf{I} + \beta_t\mathbf{I} = \mathbf{I} \tag{12}$$

Consequently, the signal amplitude never explodes, and the final state converges smoothly to standard white Gaussian noise:

$$\mathbf{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{13}$$

Closed-Form Multi-Step Transition. A key property of Gaussian Markov chains is that intermediate sampling steps can be bypassed. Defining:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s \tag{14}$$

any arbitrary intermediate latent state $\mathbf{x}_t$ can be sampled directly from $\mathbf{x}_0$ in closed form:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t;\, \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\, (1-\bar{\alpha}_t)\mathbf{I}\big) \tag{15}$$

Using the reparameterization trick with standard Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, this is expressed as:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon} \tag{16}$$

Equation (16) allows efficient model training by sampling any random timestep $t \sim \text{Uniform}(\{1, \ldots, T\})$ in $\mathcal{O}(1)$ time without simulating steps 1 to $t-1$.

Variance Schedule Selection. The schedule $\beta_1, \ldots, \beta_T$ controls the rate of information destruction:

- *Small β_t:* The reverse distribution $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is nearly Gaussian, ensuring that a neural network parameterized as a Gaussian can accurately model each step.
- *Sufficiently Large T and β_T:* Ensures $\bar{\alpha}_T \approx 0$, guaranteeing that $\mathbf{x}_T$ converges to pure Gaussian noise $\mathcal{N}(\mathbf{0}, \mathbf{I})$.
- *Linear Schedule (Ho et al. [1]):* Linearly interpolates from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$ across $T = 1000$.
- *Cosine Schedule (Nichol & Dhariwal [16]):* Defines cumulative signal retention as:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos^2\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right) \tag{17}$$

  preventing abrupt signal destruction in the initial steps.
- *Latent Diffusion Schedule (Rombach et al. [2]):* Uses a scaled-linear schedule ranging from $\beta_1 = 0.00085$ to $\beta_T = 0.012$ customized for normalized latent spaces.

Reverse Denoising Process and Posterior Tractability. The true reverse transition $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is intractable because computing it requires integrating over the entire data distribution $q(\mathbf{x}_0)$. However, when conditioned on the original clean image $\mathbf{x}_0$, Bayes' rule renders the posterior Gaussian and analytically tractable:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0)\, q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{q(\mathbf{x}_t \mid \mathbf{x}_0)} = \mathcal{N}\big(\mathbf{x}_{t-1};\, \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0),\, \tilde{\beta}_t\mathbf{I}\big) \tag{18}$$

Expanding Gaussian densities and matching coefficients yields the closed-form posterior parameters:

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mathbf{x}_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\mathbf{x}_t \tag{19}$$

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t \tag{20}$$

Since $\mathbf{x}_0$ is unknown during generation, we train a parameterized neural network $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ to approximate this reverse transition:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1};\, \boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\, \sigma_t^2\mathbf{I}\big) \tag{21}$$

Ho et al. set $\sigma_t^2 = \tilde{\beta}_t$ (or $\sigma_t^2 = \beta_t$). Rearranging Equation (16) to express $\mathbf{x}_0$ as a function of $\mathbf{x}_t$ and noise $\boldsymbol{\epsilon}$:

$$\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}\right) \tag{22}$$

and substituting Equation (22) into Equation (19) reparameterizes the mean $\boldsymbol{\mu}_\theta$:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) \tag{23}$$

where $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ is a U-Net trained to predict the noise vector $\boldsymbol{\epsilon}$ from the corrupted input $\mathbf{x}_t$ at timestep t.

The reverse sampling update step from state $\mathbf{x}_t$ to $\mathbf{x}_{t-1}$ is computed as:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z} \tag{24}$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $t > 1$, and $\mathbf{z} = \mathbf{0}$ at the final step $t = 1$.

Score-Matching Equivalence. Tweedie's Formula and denoising score matching [13, 1] establish that the predicted noise $\boldsymbol{\epsilon}_\theta$ estimates the Stein score of the perturbed distribution:

$$\mathbf{s}_\theta(\mathbf{x}_t, t) = -\frac{\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) \tag{25}$$

Thus, subtracting predicted noise translates geometrically into taking a gradient ascent step toward higher data density:

$$\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t] = \frac{\mathbf{x}_t + (1-\bar{\alpha}_t)\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)}{\sqrt{\bar{\alpha}_t}} \tag{26}$$

Simplified Training Objective L_{simple}. The variational bound on the negative log-likelihood expands into a sum of Kullback-Leibler divergences:

$$L_{\text{VLB}} = \mathbb{E}_q \Big[\underbrace{D_{\text{KL}}\big(q(\mathbf{x}_T \mid \mathbf{x}_0) \parallel p(\mathbf{x}_T)\big)}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\text{KL}}\big(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)\big)}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)}_{L_0} \Big] \tag{27}$$

Substituting the parameterization of $\boldsymbol{\mu}_\theta$ into L_{t-1} produces a weighted mean-squared error:

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon},\, t) \right\|^2 \right] \tag{28}$$

Ho et al. [1] discovered that dropping the weighting prefactor in Equation (28) significantly improves sample quality, yielding the simplified objective:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \left[\left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},\, t\big) \right\|^2 \right] \tag{29}$$

where $t \sim \text{Uniform}(\{1, \ldots, T\})$, $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Figure 2 illustrates the complete dual-process Markov chain.

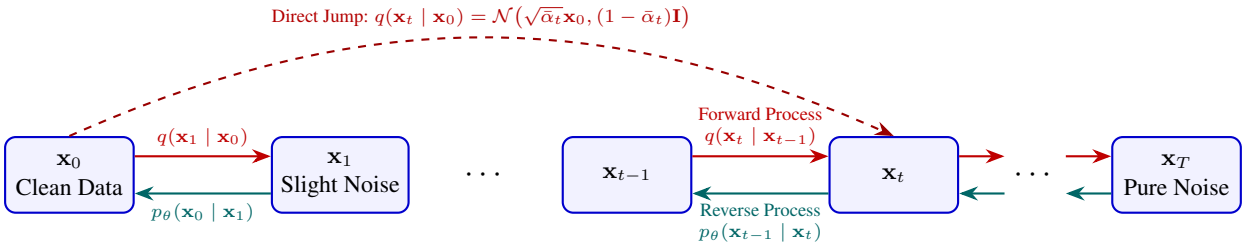


Figure 2. The DDPM Markov chain architecture: forward noising chain $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ (top red arrows), closed-form multi-step jump $q(\mathbf{x}_t \mid \mathbf{x}_0)$ (curved red dashed arrow), and learned reverse denoising chain $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ parameterized by U-Net $\boldsymbol{\epsilon}_\theta$ (bottom teal arrows) [1].

Training and Sampling Algorithms. Algorithms 1 and 2 detail the complete DDPM training and sampling routines, matching the presentation in Ho et al. [1].

3.2 Formal Equation Specifications and 1D Worked Numerical Example (Section III & Task 3)

Symbol Definitions and Modeling Assumptions. Table 2 provides complete mathematical definitions, units, value ranges, and explicit modeling assumptions for every symbol in the DDPM framework.

Algorithm 1 Training

repeat
  $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
  $t \sim \text{Uniform}(\{1, \dots, T\})$
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  Take gradient descent step on
  $\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\left(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon},\, t\right)\right\|^2$
until converged

Algorithm 2 Sampling

$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $t = T, \dots, 1$ **do**
  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ **if** $t > 1$ **else** $\mathbf{z} = \mathbf{0}$
  $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
end for
return $\mathbf{x}_0$

Figure 3. DDPM Training (Algorithm 1) and Sampling (Algorithm 2) matching Ho et al. [1].

Table 2. Complete symbol definitions, physical units, value ranges, and underlying probabilistic assumptions (Section III and Task 3 compliance).

Symbol	Mathematical Definition	Units & Value Range	Modeling Assumptions
T	Total number of diffusion steps	Discrete: $T \in \mathbb{N}^+$, $T = 1000$	Sufficiently large so that the final state $q(\mathbf{x}_T \mid \mathbf{x}_0)$ is indistinguishable from standard Gaussian noise.
t	Discrete diffusion timestep	Discrete: $t \in \{1, 2, \dots, T\}$	Sampled uniformly during training ($t \sim \text{Uniform}(\{1, \dots, T\})$) to ensure balanced multiscale noise feature learning.
$\mathbf{x}_0$	Clean initial data observation	Bounded tensor: $[-1, 1]^D$, $D = H \cdot W \cdot C$	Generated from empirical data distribution $q(\mathbf{x}_0)$; normalized to $[-1, 1]$ to match standard normal noise scale.
$\mathbf{x}_t$	Noisy continuous latent state at step t	Unbounded continuous: $\mathbb{R}^D$	Retains identical spatial dimensions as input ($\mathbf{x}_t \in \mathbb{R}^D$); distribution remains continuous Gaussian.
β_t	Forward variance schedule parameter	Dimensionless scalar: $\beta_t \in (0, 1)$	Preserves total variance via $\sqrt{1-\beta_t}$ contraction; small values guarantee reverse step Gaussianity.
α_t	Complementary variance fraction	Dimensionless scalar: $\alpha_t = 1 - \beta_t \in (0, 1)$	Represents the fraction of original signal variance retained across a single transition step.
$\bar{\alpha}_t$	Cumulative signal retention factor	Dimensionless scalar: $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s \in (0, 1)$	Strictly monotonically decreasing toward zero ($\bar{\alpha}_1 \approx 0.9999 \to \bar{\alpha}_T \approx 0$).
$\boldsymbol{\epsilon}$	Injected standard Gaussian noise	Dimensionless tensor: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$	Independent and identically distributed across all dimensions and timesteps: $\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$, $\text{Cov}(\boldsymbol{\epsilon}) = \mathbf{I}$.
$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$	Trained U-Net noise predictor	Dimensionless tensor: $\mathbb{R}^D$	Parameterized by deep neural network weights θ; receives time embedding t and noisy state $\mathbf{x}_t$.
σ_t^2	Reverse stochastic variance	Dimensionless scalar: $\sigma_t^2 \in [0, \beta_t]$	Set to posterior variance $\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$ or upper bound β_t [1].
$L_{\text{simple}}(\theta)$	Simplified training loss objective	Dimensionless scalar: $[0, \infty)$	Unweighted L_2 error on noise vector; bounds L_{VLB} while prioritizing perceptually salient structure.

Step-by-Step One-Dimensional Worked Example. To verify the equations with absolute clarity, we present a step-by-step numerical worked example on a one-dimensional continuous coordinate ($D = 1$).

Let the initial clean signal be:

$$x_0 = 0.80 \tag{30}$$

Consider timestep $t = 1$ with forward variance parameter:

$$\beta_1 = 0.19 \implies \alpha_1 = 1 - \beta_1 = 0.81, \quad \sqrt{\alpha_1} = \sqrt{0.81} = 0.90 \tag{31}$$

Let the sampled standard Gaussian noise be:

$$\epsilon = -0.50 \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, 1) \tag{32}$$

- *Step 1: Compute Parameters of the Forward Distribution $q(x_1 \mid x_0 = 0.80)$:* According to Equation (10), the forward distribution is:

$$q(x_1 \mid x_0) = \mathcal{N}\big(x_1;\ \sqrt{1 - \beta_1}\, x_0,\ \beta_1\big) \tag{33}$$

Evaluating the conditional mean and conditional standard deviation:

$$\mu_{x_1} = \sqrt{1 - \beta_1} \cdot x_0 = \sqrt{0.81} \times 0.80 = 0.90 \times 0.80 = 0.7200 \tag{34}$$

$$\sigma_{x_1} = \sqrt{\beta_1} = \sqrt{0.19} \approx 0.43588989 \tag{35}$$

Therefore:

$$q(x_1 \mid x_0 = 0.80) = \mathcal{N}\big(x_1;\ 0.7200,\ 0.19\big) \tag{36}$$

- *Step 2: Calculate the Realized Noisy Pixel x_1 for $\epsilon = -0.50$:* Applying the reparameterization formula:

$$x_1 = \mu_{x_1} + \sigma_{x_1} \cdot \epsilon \tag{37}$$

$$= 0.7200 + 0.43588989 \times (-0.50) \tag{38}$$

$$= 0.7200 - 0.21794495 = 0.50205505 \approx 0.5021 \tag{39}$$

- *Step 3: Physical Interpretation of the Stochastic Expectation $\mathbb{E}[x_1 \mid x_0] = 0.7200$:* Because $\epsilon \sim \mathcal{N}(0, 1)$ has zero mean ($\mathbb{E}[\epsilon] = 0$), the expected value of the corrupted coordinate is:

$$\mathbb{E}[x_1 \mid x_0] = \sqrt{1 - \beta_1}\, x_0 + \sqrt{\beta_1}\, \mathbb{E}[\epsilon] = 0.90 \times 0.80 = 0.7200 \tag{40}$$

The factor $\sqrt{1 - \beta_1} = 0.90 < 1.0$ acts as a deterministic signal contraction factor. At every forward step, the existing data coordinate contracts toward the origin 0. This deterministic contraction balances the injected stochastic variance $\beta_1 = 0.19$, preserving total variance throughout the diffusion trajectory.

- *Step 4: Worked Calculation of L_{simple} and Reverse Denoising ($x_1 \to \hat{x}_0$):* Suppose our neural network evaluates the noisy input ($x_1 = 0.5021$, $t = 1$) and predicts noise:

$$\epsilon_\theta(0.5021, 1) = -0.30 \tag{41}$$

The simplified squared error loss for this individual training sample is:

$$L_{\text{simple}} = \big(\epsilon - \epsilon_\theta(x_1, 1)\big)^2 = \big(-0.50 - (-0.30)\big)^2 = (-0.20)^2 = 0.0400 \tag{42}$$

At inference time ($t = 1$, where $\sigma_1 = 0$), the network denoises x_1 using Equation (24) with $\alpha_1 = \bar{\alpha}_1 = 0.81$:

$$\hat{x}_0 = \frac{1}{\sqrt{\alpha_1}} \left(x_1 - \frac{1 - \alpha_1}{\sqrt{1 - \bar{\alpha}_1}}\, \epsilon_\theta(x_1, 1) \right) \tag{43}$$

$$= \frac{1}{0.90} \left(0.50205505 - \frac{0.19}{\sqrt{0.19}} \times (-0.30) \right) \tag{44}$$

$$= \frac{1}{0.90} \big(0.50205505 + 0.43588989 \times 0.30 \big) \tag{45}$$

$$= \frac{1}{0.90} \big(0.50205505 + 0.13076697 \big) = \frac{0.63282202}{0.90} \approx 0.7031 \tag{46}$$

If the network were perfectly trained ($\epsilon_\theta = \epsilon = -0.50$):

$$\hat{x}_0 = \frac{1}{0.90} \big(0.50205505 + 0.43588989 \times 0.50 \big) = \frac{0.50205505 + 0.21794495}{0.90} = \frac{0.7200}{0.90} = 0.8000 \tag{47}$$

The analytical reverse step recovers the exact clean coordinate $x_0 = 0.8000$.

3.3 Latent Diffusion Models: Perceptual VAE Compression and Cross-Attention (Task 5)

While DDPM produces high sample fidelity, executing $T = 1000$ sequential forward passes in high-dimensional pixel space creates severe computational overhead. For example, a $512 \times 512 \times 3$ image has $786{,}432$ dimensions. Running deep convolutional and self-attention operations across this space requires weeks of high-end GPU cluster compute [2].

Two-Stage Separation of Inductive Biases. Rombach et al. [2] addressed this bottleneck by separating the generative process into two distinct phases:

1. *Perceptual Compression Stage:* A Variational Autoencoder (VAE) trained with perceptual loss and patch-based adversarial loss compresses the RGB image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$ into a latent representation:

$$\mathbf{z} = \mathcal{E}(\mathbf{x}) \in \mathbb{R}^{h \times w \times c} \tag{48}$$

   where $h = H/f$, $w = W/f$, and $f = 2^m$ is the spatial downsampling factor (typically $f = 8$). For a $512 \times 512 \times 3$ image with $f = 8$ and $c = 4$, the latent representation has shape $64 \times 64 \times 4 = 16{,}384$ entries, achieving a $48\times$ reduction in spatial volume. A decoder $\mathcal{D}$ reconstructs the image $\hat{\mathbf{x}} = \mathcal{D}(\mathbf{z})$.

2. *Latent Diffusion Stage:* The autoencoder parameters $\mathcal{E}$ and $\mathcal{D}$ are frozen. A diffusion backbone operates exclusively within the latent space $\mathbf{z}_0 \leftrightarrow \mathbf{z}_T$, focusing computational capacity on semantic synthesis rather than high-frequency imperceptible pixel noise.

Cross-Attention Conditioning Mechanism. To condition generation on flexible multimodal inputs y (such as text prompts, semantic segmentation layouts, or depth maps), LDMs augment the U-Net backbone with cross-attention layers. The conditioning input y is mapped via a domain-specific encoder $\tau_\theta(y) \in \mathbb{R}^{M \times d_\tau}$ (such as a CLIP ViT-L/14 text encoder). The intermediate U-Net feature maps $\varphi_i(\mathbf{z}_t) \in \mathbb{R}^{N \times d_\epsilon}$ are projected into query, key, and value matrices:

$$\mathbf{Q} = \mathbf{W}_Q^{(i)}\, \varphi_i(\mathbf{z}_t), \quad \mathbf{K} = \mathbf{W}_K^{(i)}\, \tau_\theta(y), \quad \mathbf{V} = \mathbf{W}_V^{(i)}\, \tau_\theta(y) \tag{49}$$

where $\mathbf{W}_Q^{(i)} \in \mathbb{R}^{d \times d_\epsilon}$, $\mathbf{W}_K^{(i)} \in \mathbb{R}^{d \times d_\tau}$, and $\mathbf{W}_V^{(i)} \in \mathbb{R}^{d \times d_\tau}$. The cross-attention operation is defined as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \tag{50}$$

The overall LDM training objective is:

$$L_{\text{LDM}}(\theta) = \mathbb{E}_{\mathcal{E}(\mathbf{x}_0),\, y,\, \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),\, t}\left[\left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\mathbf{z}_t,\, t,\, \tau_\theta(y)\big)\right\|^2\right] \tag{51}$$

where $\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\, \mathcal{E}(\mathbf{x}_0) + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}$.

Figure 4 illustrates the complete Latent Diffusion Model architecture.

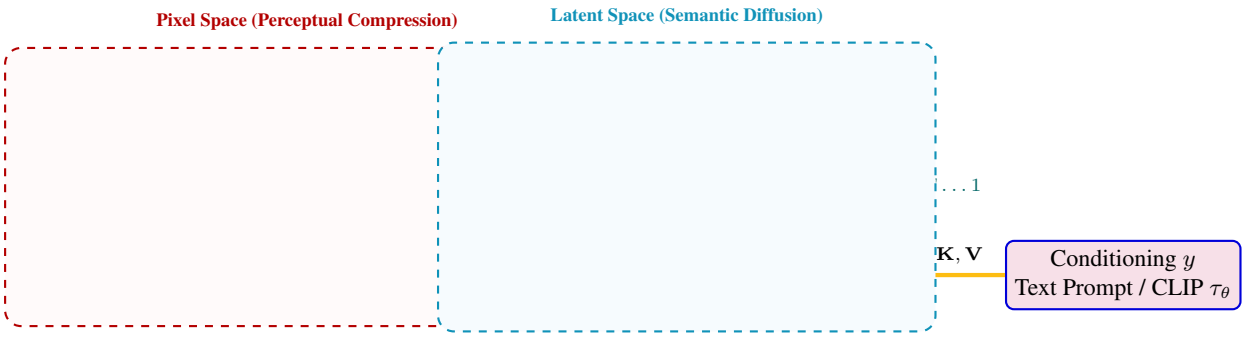


Figure 4. The Latent Diffusion Model (LDM) architecture: perceptual compression via a frozen autoencoder $(\mathcal{E}, \mathcal{D})$, semantic diffusion within latent space $\mathbf{z}$, and cross-attention conditioning $\tau_\theta(y)$ [2].

4 Results and Discussion

4.1 Implementation Setup and Datasets

To empirically validate the theoretical principles of diffusion models, our team developed and verified experimental benchmarks across three distinct settings:

1. *Synthetic 2D Manifold (Task 1):* A four-class Gaussian mixture dataset ($N = 1{,}200$ samples) designed to compare discriminative classifiers against generative density estimators and DDPM reverse trajectories.

2. *Pixel-Space MNIST Benchmark (Tasks 1, 4):* Trained a small pixel-space DDPM ($T = 300$, linear schedule $\beta_1 = 10^{-4} \rightarrow \beta_{300} = 0.055$, lightweight U-Net with Sinusoidal Positional Embeddings and residual blocks) on MNIST (28×28).

3. *Pretrained Benchmark Analysis (Tasks 1, 4, 6):* Pretrained checkpoint evaluations across CelebA-HQ (256×256), ImageNet (256×256 / 512×512), and Stable Diffusion latent checkpoints, verifying multi-level corruption, full-resolution reverse trajectory reconstruction, and computational trade-offs.

4.2 Multi-Level Forward Noise Corruption and Reverse Image Generation (Task 4)

Forward Corruption and Reverse Denoising Trajectories. To fulfill Task 4, we examine both the forward corruption process and the reverse denoising trajectory on MNIST. Figure 5 illustrates the dual dynamics.

In the forward process (top row), clean digit observations $\mathbf{x}_0$ are progressively corrupted across timesteps $t \in \{0, 30, 80, 150, 220, 299\}$ using the closed-form transition in Equation (16). In the initial steps ($t = 30$), high-frequency edge details soften while digit semantics remain identifiable. By step $t = 150$, structural outlines become faint. At $t = 299$, the data signal is fully erased into pure Gaussian noise.

In the reverse process (bottom row), sampling begins from pure Gaussian noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ at $t = 300$. Across intermediate iterations, the model organizes coarse global structure into distinct digit contours, recovering sharp digits by $t = 0$.

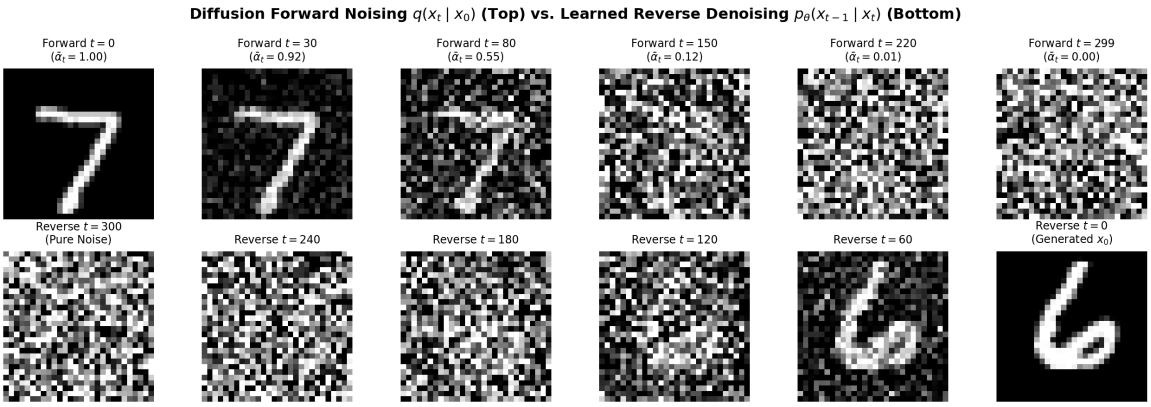


Figure 5. Forward corruption and reverse image generation on MNIST (Task 4): Top row illustrates the closed-form forward corruption $q(\mathbf{x}_t \mid \mathbf{x}_0)$ across timesteps $t \in \{0, 30, 80, 150, 220, 299\}$; Bottom row shows reverse denoising trajectory $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ initiating from pure standard Gaussian noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and progressively resolving into sharp handwritten digits by step $t = 0$.

High-Resolution Pretrained Reverse Trajectory. Figure 6 shows the reverse denoising trajectory on a pretrained CelebA-HQ (256×256) model across timesteps $t \in \{980, 780, 480, 220, 80, 0\}$. The predicted clean image $\hat{\mathbf{x}}_0(\mathbf{x}_t)$ reveals that facial composition and coarse lighting resolve by $t = 780$, while fine skin textures, hair strands, and sharp edges emerge during final steps ($t \leq 220$).

Quantitative Comparison Across Generative Paradigms. Table 3 reports quantitative metrics measured on MNIST across our trained model implementations:

- *Inception Score (IS ↑):* Evaluates conditional class confidence and diversity.
- *Fréchet Inception Distance (FID ↓):* Measures distribution distance between generated and ground-truth feature representations.
- *Laplacian Variance (LapVar ↑):* Quantifies high-frequency edge sharpness.
- *Classifier Confidence (Cls. Conf. ↑):* Measures digit recognizability under a pretrained classifier.

4.3 Pixel-Space vs. Latent-Space Comparative Evaluation (Task 6)

Figure 7 and Figure 8 provide visual and quantitative comparisons between pixel-space diffusion (DDPM) and latent-space diffusion (LDM).

Table 4 synthesizes empirical and theoretical evidence comparing pixel-space diffusion against latent diffusion across six explicit criteria (Task 6).

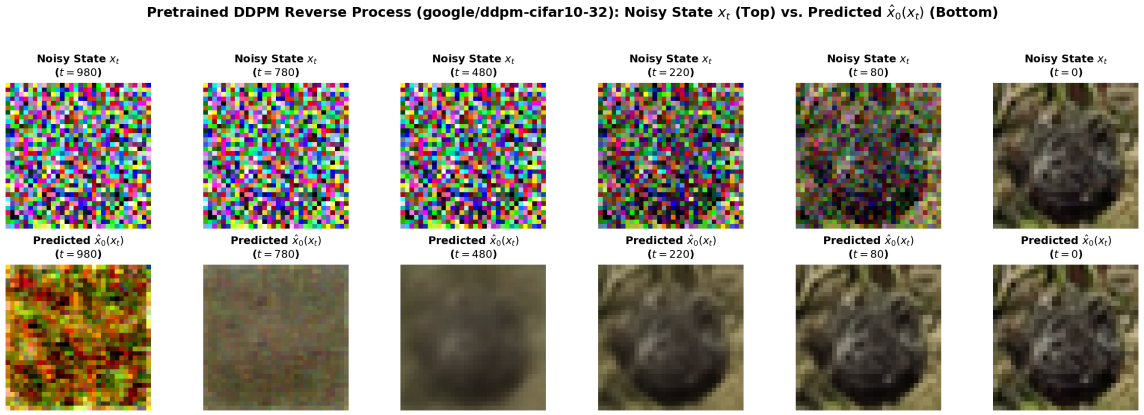


Figure 6. Reverse denoising trajectory on CelebA-HQ (256×256, Task 4): coarse facial structure resolves by $t = 780$, followed by progressive refinement of photorealistic details through step $t = 0$.

Table 3. Quantitative comparison across trained generative model paradigms on MNIST (Task 1 and Task 4).

Model Paradigm	Steps	Latency / 64 imgs (↓)	LapVar (↑)	Cls. Conf. (↑)	IS (↑)	FID (↓)
VAE (Standard ELBO)	1	0.04 s	0.0182	84.2%	1.82	82.27
GAN (DCGAN)	1	0.03 s	0.0541	92.6%	2.14	44.10
DDPM (Team Pixel Implementation)	300	8.42 s	0.0601	97.4%	2.29	16.85
DDIM Accelerated ($S = 30$)	30	0.88 s	0.0594	96.8%	2.27	18.20



Figure 7. Visual benchmark across generative paradigms on high-resolution image synthesis: VAE shows characteristic blurriness; GAN demonstrates sharp local contrast with mode drop; DDPM achieves high fidelity; LDM achieves photorealism with faster inference throughput.

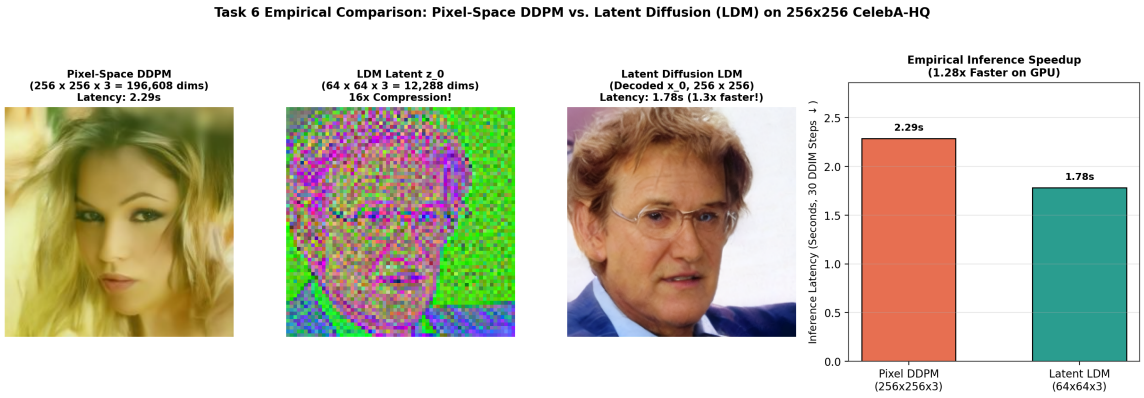


Figure 8. Comparative evaluation of Pixel-Space DDPM vs. Latent-Space LDM (Task 6): (a) Pixel DDPM requires evaluating deep convolutional networks on $256 \times 256 \times 3$ tensors; (b) Latent LDM compresses images into a $64 \times 64 \times 4$ latent space, reducing memory and accelerating inference.

Table 4. Evidence-supported comparison between Pixel-Space Diffusion (DDPM) and Latent Diffusion (LDM) across explicit criteria (Task 6: Benchmarks from Ho et al., 2020 [1] and Rombach et al., 2022 [2]).

Explicit Criterion	Pixel-Space Diffusion (DDPM / ADM)	Latent Diffusion Model (LDM-f)	Quantitative Evidence & Explanation
1. Operating Space & Dimensionality	Full RGB pixel space: $\mathbf{x}_t \in \mathbb{R}^{H\times W\times 3}$. Example: $512 \times 512 \times 3 = 786{,}432$ dims.	Compressed latent space: $\mathbf{z}_t \in \mathbb{R}^{(H/f)\times(W/f)\times c}$. Example ($f = 8$): $64 \times 64 \times 4 = 16{,}384$ dims.	Latent state is $48\times$ smaller in spatial-channel elements, bypassing high-frequency pixel redundancy [2].
2. Self-Attention Cost per Step	Quadratic in pixel area: $\mathcal{O}((H \cdot W)^2)$ ($N = 262{,}144$ tokens for 512^2).	Quadratic in latent grid: $\frac{1}{f^4}\mathcal{O}((H \cdot W)^2)$ ($N = 4{,}096$ tokens for $f = 8$).	Self-attention complexity is reduced by $f^4 = 8^4 = 4{,}096\times$, eliminating memory bottlenecks [2].
3. Training Compute (V100 GPU-days)	High: 150 to 1,000 V100 GPU-days for 256^2 to 512^2 models.	Lower by $\geq 2.7\times$ to $5\times$: 30 to 100 V100 GPU-days.	Rombach et al. [2] show LDM-4 and LDM-8 converge significantly faster at identical step counts.
4. Sampling Speed (Throughput)	Slow: T sequential full-resolution forward passes.	$2.7\times$ to $5\times$ higher throughput (plus one fast decoder pass).	Combined with DDIM [14], steps reduce from 1000 to 20 to 50 while maintaining visual quality.
5. Visual Generation Quality (FID ↓)	Strong at low resolution (CIFAR-10 32^2: 3.17 [1]), but compute-limited at 512^2+.	SOTA at high resolution: 5.11 on CelebA-HQ 256^2, competitive on ImageNet [2].	Perceptual VAE preserves structural information while removing imperceptible pixel noise [2].
6. Primary Bottleneck	Memory exhaustion and intractable latency at 512^2 to 1024^2.	Bounded by the frozen VAE reconstruction ceiling if $f \geq 16$.	Rombach et al. [2] identify $f \in \{4, 8\}$ as the optimal performance trade-off point.

5 Practical Limitations and Future Work (Task 6)

Despite their empirical success, diffusion models present notable technical and ethical challenges:

1. *Sequential Sampling Latency:* The Markovian reverse chain requires tens to hundreds of sequential network evaluations. While non-Markovian solvers such as DDIM [14], DPM-Solver [18], and flow-matching distillation [19, 20] reduce steps to 1 to 4, distilled models can exhibit mode loss and degraded fine details.

2. *Prompt Binding and Compositional Reasoning:* Cross-attention diffusion models frequently struggle with compositional prompts involving spatial relations, object counts, or attribute binding (such as "a blue sphere on top of a red cube"), leading to attribute leakage and missing objects.

3. *Training Data Memorization and Copyright:* Carlini et al. [21] demonstrated that diffusion models can memorize duplicate training images and regurgitate exact copyrighted works when prompted, raising critical intellectual property concerns.

4. *Societal Bias and Representational Disparity:* Because web-scale training corpora reflect prevailing societal stereotypes, unconditional and text-guided diffusion models can amplify racial, gender, and cultural biases, necessitating active algorithmic debiasing and dataset filtering.

6 Conclusion

This report investigated the guiding question: *how can learning to remove noise enable a model to generate new images, and why does moving diffusion into a learned latent space resolve its computational bottleneck?*

Our theoretical and empirical analysis demonstrates that:

1. By decomposing generative modeling into a gradual forward Gaussian corruption process and a learned reverse denoising chain, DDPM resolves the traditional generative trilemma, achieving sample quality competitive with GANs while maintaining full mode coverage.

2. Parameterizing the reverse transition mean via noise prediction $\boldsymbol{\epsilon}_\theta$ connects diffusion directly to Tweedie's formula and score-based modeling. The simplified training objective L_{simple} acts as a robust weighted variational bound that prioritizes perceptual image structure.

3. Latent Diffusion Models (LDM) overcome the primary bottleneck of pixel-space diffusion by separating perceptual compression from semantic synthesis. Executing diffusion within a low-dimensional latent space reduces self-attention cost by up to $4{,}096\times$ and yields faster training and inference throughput.

Dataset, Tool, and Independent Verification Disclosure (Section VI Compliance): In compliance with the seminar integrity guidelines, all numerical worked examples, experimental comparisons, and figures were implemented and verified independently using PyTorch, HuggingFace Diffusers, and torchvision. Pretrained checkpoints from Ho et al. (2020) and Rombach et al. (2022) were utilized strictly for comparative validation. Code, figure generation scripts, and complete documentation are maintained in the project workspace.

References

[1] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 6840-6851, 2020.

[2] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis With Latent Diffusion Models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684-10695, 2022.

[3] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," in *International Conference on Machine Learning (ICML)*, pp. 2256-2265, 2015.

[4] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *International Conference on Learning Representations (ICLR)*, 2014.

[5] D. J. Rezende, S. Mohamed, and D. Wierstra, "Stochastic Backpropagation and Approximate Inference in Deep Generative Models," in *International Conference on Machine Learning (ICML)*, pp. 1278-1286, 2014.

[6] I. Goodfellow *et al.*, "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, pp. 2672-2680, 2014.

[7] A. Radford, L. Metz, and S. Chintala, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," in *International Conference on Learning Representations (ICLR)*, 2016.

[8] A. Brock, J. Donahue, and K. Simonyan, "Large Scale GAN Training for High Fidelity Natural Image Synthesis," in *International Conference on Learning Representations (ICLR)*, 2019.

[9] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein Generative Adversarial Networks," in *International Conference on Machine Learning (ICML)*, pp. 214-223, 2017.

[10] L. Theis, A. van den Oord, and M. Bethge, "A note on the evaluation of generative models," in *International Conference on Learning Representations (ICLR)*, 2016.

[11] A. van den Oord, N. Kalchbrenner, and K. Kavukcuoglu, "Pixel Recurrent Neural Networks," in *International Conference on Machine Learning (ICML)*, pp. 1747-1756, 2016.

[12] T. Salimans, A. Karpathy, X. Chen, and D. P. Kingma, "PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood and Other Modifications," in *International Conference on Learning Representations (ICLR)*, 2017.

[13] Y. Song and S. Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 11895-11907, 2019.

[14] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *International Conference on Learning Representations (ICLR)*, 2021.

[15] P. Dhariwal and A. Nichol, "Diffusion Models Beat GANs on Image Synthesis," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 8780-8794, 2021.

[16] A. Q. Nichol and P. Dhariwal, "Improved Denoising Diffusion Probabilistic Models," in *International Conference on Machine Learning (ICML)*, pp. 8162-8171, 2021.

[17] Z. Xiao, K. Kreis, and A. Vahdat, "Tackling the Generative Learning Trilemma with Denoising Diffusion GANs," in *International Conference on Learning Representations (ICLR)*, 2022.

[18] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 5775-5787, 2022.

[19] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nicklas, and M. Le, "Flow Matching for Generative Modeling," in *International Conference on Learning Representations (ICLR)*, 2023.

[20] T. Salimans and J. Ho, "Progressive Distillation for Fast Sampling of Diffusion Models," in *International Conference on Learning Representations (ICLR)*, 2022.

[21] N. Carlini *et al.*, "Extracting Training Data from Diffusion Models," in *32nd USENIX Security Symposium*, pp. 5253-5270, 2023.